

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

TA – Tarea de práctica fuera de clases

Extender proyecto ASP.NET Core MVC

MiPrimerMVC-TA

Asignatura: Aplicaciones Web

Estudiante: Mariscal Cabrera Jaime Josué

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Periodo Académico: 2025-2026 SPA

Repositorio: <https://github.com/Grinjoww/MiPrimerMVC-TA>

Rama: feature/practica-aspnet

Quevedo, Ecuador

18 de junio de 2026

1. Objetivo

Extender el proyecto **MiPrimerMVC** desarrollado en ASP.NET Core MVC, agregando una tabla HTML para mostrar los productos registrados, una acción **Delete** con confirmación por POST y protección CSRF, además de la validación del lado del cliente y del servidor. También se considera la entrega del proyecto en la rama **feature/practica-aspnet** y la actualización del archivo **README.md** con las instrucciones de ejecución.

1.1. Requisitos previos

Para trabajar sobre esta práctica se requiere contar con los siguientes elementos:

- Proyecto base **MiPrimerMVC**.
- .NET SDK 8.0 o superior instalado.
- Visual Studio 2022 o Visual Studio Code.
- Navegador web para probar el funcionamiento.
- Git configurado para subir los cambios al repositorio.

La versión del SDK puede verificarse desde la terminal con el comando **dotnet -version**.

2. Paso 1 – Tabla HTML con lista de productos

Como primera extensión del proyecto, se modificó la vista **Index.cshtml** para mostrar los productos en una tabla HTML. La tabla presenta los datos principales del producto, como identificador, nombre, descripción, precio, stock y correo del proveedor.

Además, se agregó una columna de **Acciones**, donde se ubica el botón **Eliminar**. Este botón permite acceder a la vista de confirmación antes de borrar el producto.

```
1  @model IEnumerable<MiPrimerMVC.Models.Producto>
2  @{
3      ViewData["Title"] = "Listado de Productos";
4  }
5
6  <h1>Productos</h1>
7
8  <p>
9      <a asp-action="Create" class="btn btn-primary">Crear nuevo producto</a>
10 </p>
11
12 @if (!Model.Any())
13 {
14     <div class="alert alert-info">Aun no hay productos registrados.</div>
15 }
16 else
17 {
18     <table class="table table-striped table-bordered">
19         <thead class="table-dark">
20             <tr>
21                 <th>Id</th>
22                 <th>@Html.DisplayNameFor(m => m.First().Nombre)</th>
23                 <th>@Html.DisplayNameFor(m => m.First().Descripcion)</th>
```

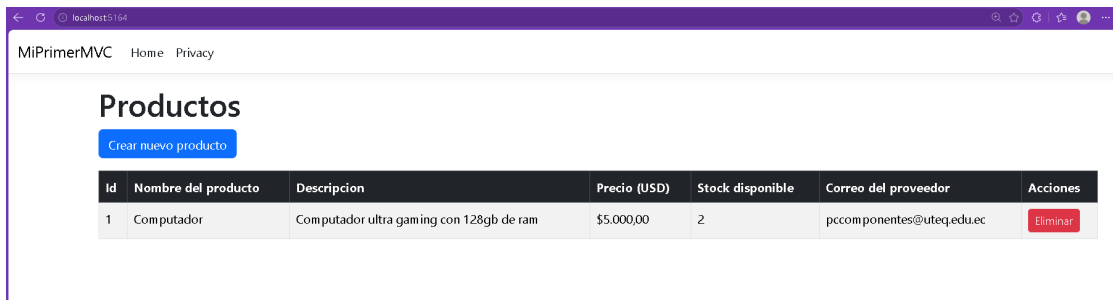
```

24         <th>@Html.DisplayNameFor(m => m.First().Precio)</th>
25         <th>@Html.DisplayNameFor(m => m.First().Stock)</th>
26         <th>@Html.DisplayNameFor(m =>
27             m.First().CorreoProveedor)</th>
28         <th>Acciones</th>
29     </tr>
30 </thead>
31 <tbody>
32     @foreach (var item in Model)
33     {
34         <tr>
35             <td>@item.Id</td>
36             <td>@item.Nombre</td>
37             <td>@item.Descripcion</td>
38             <td>@item.Precio.ToString("C")</td>
39             <td>@item.Stock</td>
40             <td>@item.CorreoProveedor</td>
41             <td>
42                 <a asp-action="Delete" asp-route-id="@item.Id"
43                     class="btn btn-sm btn-danger">Eliminar</a>
44             </td>
45         </tr>
46     }
47 </tbody>
48 </table>

```

Listing 1: Views/Producto/Index.cshtml

Con este cambio, el listado deja de mostrarse como información suelta y pasa a tener una presentación más ordenada. La tabla también facilita ubicar las acciones disponibles para cada producto.



Id	Nombre del producto	Descripción	Precio (USD)	Stock disponible	Correo del proveedor	Acciones
1	Computador	Computador ultra gaming con 128gb de ram	\$5.000,00	2	pccomponentes@uteq.edu.ec	Eliminar

Figura 1: Listado de productos en tabla HTML con la columna Acciones

3. Paso 2 – Acción Delete con confirmación POST y CSRF

Para implementar la eliminación, se agregaron dos acciones en el controlador `ProductoController`. La primera acción, de tipo `GET`, muestra la página de confirmación. La segunda acción, de tipo `POST`, realiza la eliminación real del producto.

La acción `POST` se protegió con `[ValidateAntiForgeryToken]`, lo cual permite validar el token enviado desde el formulario y evitar solicitudes externas no autorizadas.

```

1 // GET: /Producto/Delete/5 -> muestra la pagina de confirmacion
2 [HttpGet]

```

```

3 public IActionResult Delete(int id)
4 {
5     var producto = _productos.FirstOrDefault(p => p.Id == id);
6     if (producto == null)
7     {
8         return NotFound();
9     }
10
11     return View(producto);
12 }
13
14 // POST: /Producto/Delete/5 -> elimina realmente el producto
15 // ActionName("Delete") permite que el formulario apunte a /Producto/Delete
16 // ValidateAntiForgeryToken exige el token CSRF generado en la vista.
17 [HttpPost, ActionName("Delete")]
18 [ValidateAntiForgeryToken]
19 public IActionResult DeleteConfirmed(int id)
20 {
21     var producto = _productos.FirstOrDefault(p => p.Id == id);
22     if (producto != null)
23     {
24         _productos.Remove(producto);
25     }
26
27     return RedirectToAction(nameof(Index));
28 }

```

Listing 2: Controllers/ProductoController.cs - Acciones Delete

La vista Delete.cshtml muestra los datos del producto seleccionado antes de eliminarlo. De esta manera, el usuario puede confirmar si realmente desea borrar el registro o cancelar la acción.

```

1 @model MiPrimerMVC.Models.Producto
2 @{
3     ViewData["Title"] = "Eliminar Producto";
4 }
5
6 <h1>Eliminar Producto</h1>
7 <h4 class="text-danger">¿Está seguro de que desea eliminar este
8     producto?</h4>
9
10 <hr />
11 <dl class="row">
12     <dt class="col-sm-3">@Html.DisplayNameFor(m => m.Nombre)</dt>
13     <dd class="col-sm-9">@Model.Nombre</dd>
14
15     <dt class="col-sm-3">@Html.DisplayNameFor(m => m.Descripcion)</dt>
16     <dd class="col-sm-9">@Model.Descripcion</dd>
17
18     <dt class="col-sm-3">@Html.DisplayNameFor(m => m.Precio)</dt>
19     <dd class="col-sm-9">@Model.Precio.ToString("C")</dd>
20
21     <dt class="col-sm-3">@Html.DisplayNameFor(m => m.Stock)</dt>
22     <dd class="col-sm-9">@Model.Stock</dd>
23
24     <dt class="col-sm-3">@Html.DisplayNameFor(m => m.CorreoProveedor)</dt>
25     <dd class="col-sm-9">@Model.CorreoProveedor</dd>
26 </dl>

```

```
27
28 <form asp-action="Delete" method="post" asp-antiforgery="false">
29     @Html.AntiForgeryToken()
30     <input type="hidden" asp-for="Id" />
31
32     <button type="submit" class="btn btn-danger">Eliminar</button>
33     <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
34 </form>
```

Listing 3: Views/Producto/Delete.cshtml

El formulario se envía por POST y contiene el token antifalsificación generado con `@Html.AntiForgeryToken()`. Este token es revisado por el controlador antes de aceptar la eliminación.

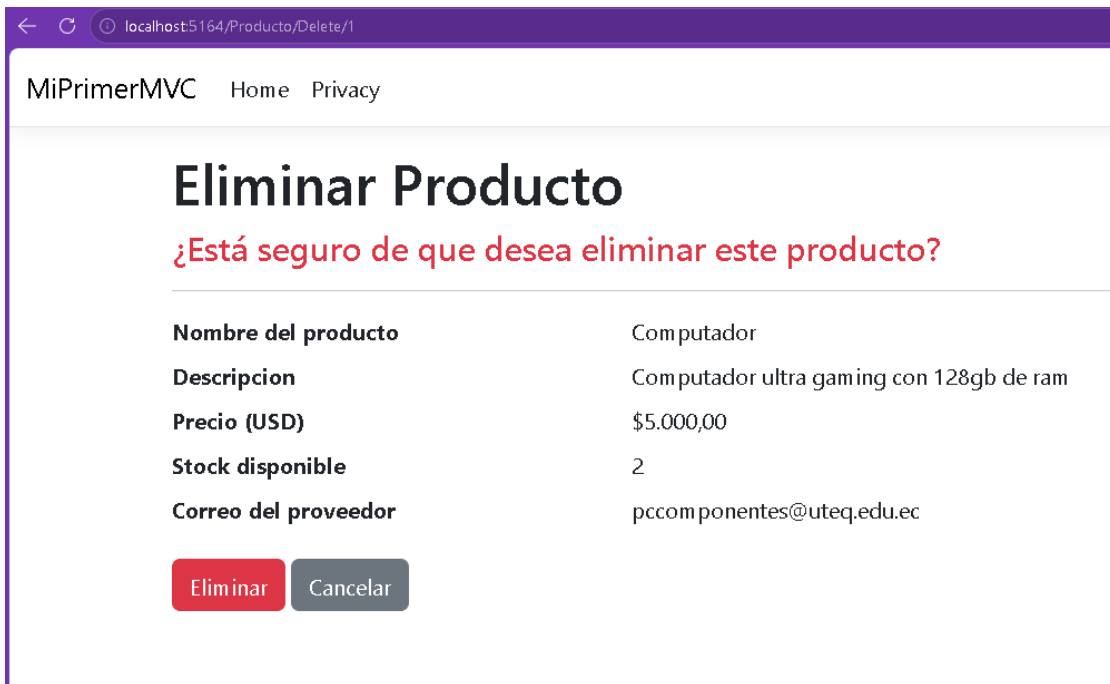


Figura 2: Página de confirmación antes de eliminar el producto

```

ProductoController.cs
Controllers > ProductoController.cs > ...
1  using Microsoft.AspNetCore.Mvc;
2  using MiPrimerMVC.Models;
3
4  namespace MiPrimerMVC.Controllers
5  {
6      // 0 references
7      public class ProductoController : Controller
8      {
9          // 7 references
10         // Almacenamiento en memoria. La practica no exige persistencia.
11         private static readonly List<Producto> _productos = new();
12
13         // GET: /Producto
14         [HttpGet]
15         public IActionResult Index()
16         {
17             return View(_productos);
18         }
19
20         // GET: /Producto/Create
21         [HttpGet]
22         public IActionResult Create()
23         {
24             return View();
25         }
26
27         // POST: /Producto/Create
28         [HttpPost]
29         [ValidateAntiForgeryToken]
30         public IActionResult Create(Producto producto)
31         {
32             // 0 references
33             // Validacion del lado del SERVIDOR.
34             if (!ModelState.IsValid)
35             {
36                 return View(producto);
37             }
38
39             producto.Id = _productos.Count == 0 ? 1 : _productos.Max(p => p.Id) + 1;
40             _productos.Add(producto);
41
42             return RedirectToAction(nameof(Index));
43         }
44
45         // GET: /Producto/Delete/{Id}
46         [HttpGet]
47         public IActionResult Delete(int id)
48         {
49             // 0 references
50             // Validacion del lado del SERVIDOR.
51             if (!ModelState.IsValid)
52             {
53                 return View(id);
54             }
55
56             // Eliminar producto
57             _productos.Remove(_productos.FirstOrDefault(p => p.Id == id));
58
59             return RedirectToAction(nameof(Index));
60         }
61
62         // GET: /Producto/DeleteConfirmed/{Id}
63         [HttpGet]
64         public IActionResult DeleteConfirmed(int id)
65         {
66             // 0 references
67             // Validacion del lado del SERVIDOR.
68             if (!ModelState.IsValid)
69             {
70                 return View(id);
71             }
72
73             // Eliminar producto
74             _productos.Remove(_productos.FirstOrDefault(p => p.Id == id));
75
76             return RedirectToAction(nameof(Index));
77         }
78     }
79 }

```

Figura 3: Código del controlador con las acciones Delete y DeleteConfirmed

```

53 <dt class="col-sm-3">Stock disponible</dt>
54 <dd class="col-sm-9">2</dd>
55
56 <dt class="col-sm-3">Correo del proveedor</dt>
57 <dd class="col-sm-9">pccomponentes@uteq.edu.ec</dd>
58 </dl>
59
60 <form method="post" action="/Producto/Delete/1">
61     <input name="RequestVerificationToken" type="hidden" value="CfDJ8IUyZdAF_Z5IvU4H5ajFHZdvOBn7aqSfkNyn_bZzLDpMeuJ2LPe575">
62     <input type="hidden" data-val="true" data-val-required="The Id field is required." id="Id" name="Id" value="1" />
63
64     <button type="submit" class="btn btn-danger">Eliminar</button>
65     <a class="btn btn-secondary" href="/">Cancelar</a>
66 </form>
67
68 </main>
69 </div>
70
71 <footer b-i33tsj388z class="border-top footer text-muted">
72     <div b-i33tsj388z class="container">
73         &copy; 2026 - MiPrimerMVC - <a href="/Home/Privacy">Privacy</a>

```

Figura 4: Formulario de eliminación con token CSRF generado en el HTML

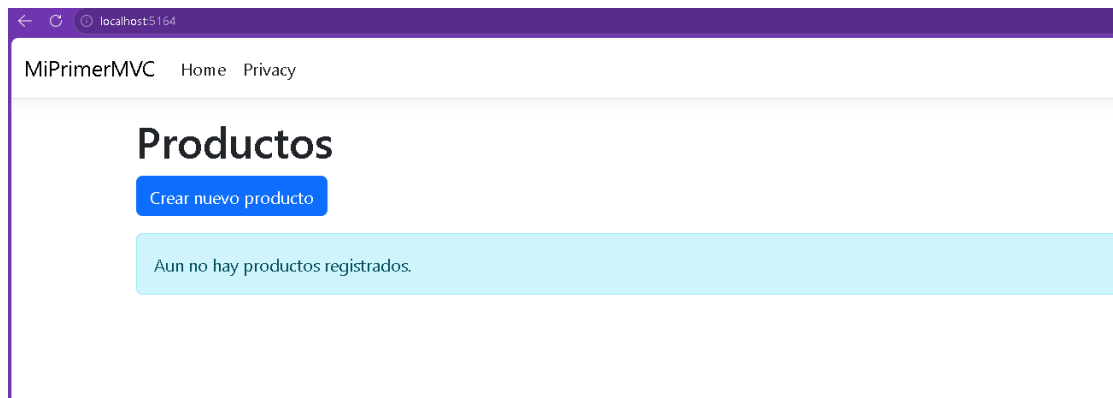


Figura 5: Listado actualizado luego de eliminar un producto

4. Paso 3 – Validación del lado del cliente y del servidor

La validación del lado del servidor se mantiene en la acción **Create** del controlador. Esto se realiza mediante `ModelState.IsValid`, que revisa si los datos enviados cumplen con las reglas establecidas en el modelo **Producto**.

```
1 [HttpPost]
2 [ValidateAntiForgeryToken]
3 public IActionResult Create(Producto producto)
4 {
5     // Validacion del lado del SERVIDOR.
6     if (!ModelState.IsValid)
7     {
8         return View(producto);
9     }
10
11     producto.Id = _productos.Count == 0 ? 1 : _productos.Max(p => p.Id) + 1;
12     _productos.Add(producto);
13
14     return RedirectToAction(nameof(Index));
15 }
```

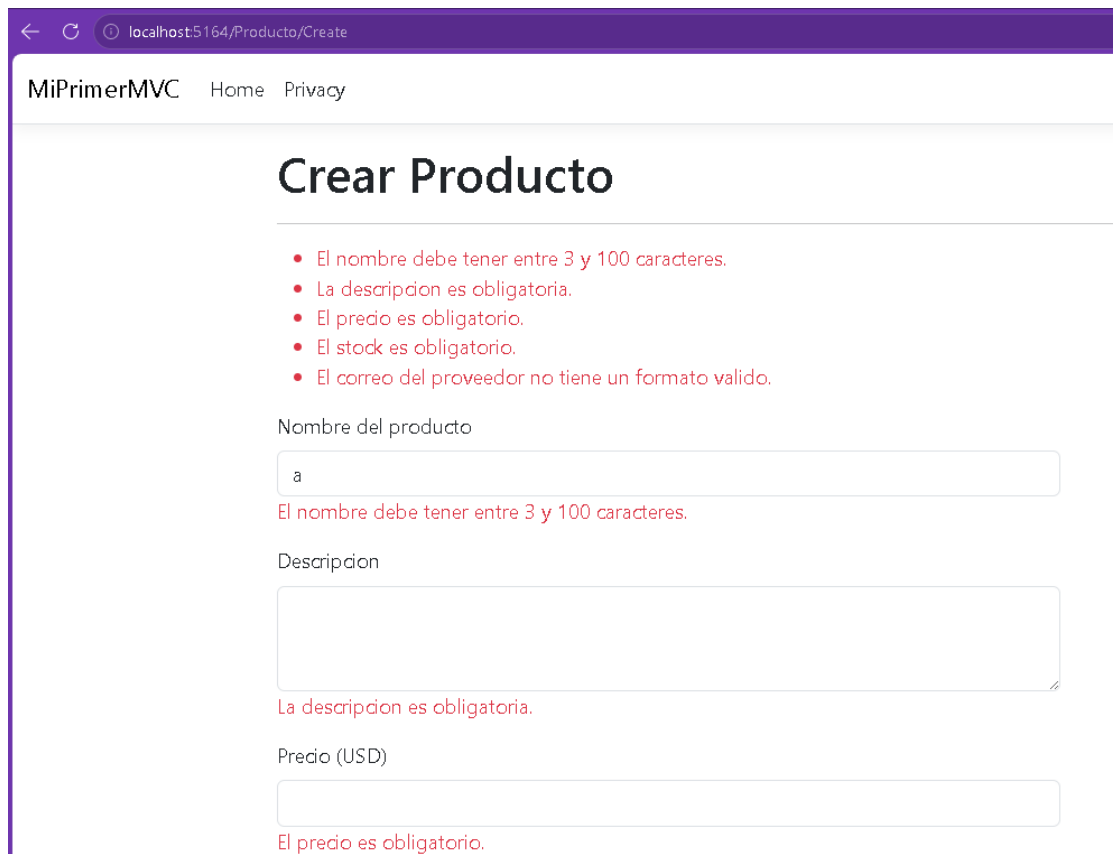
Listing 4: Validación del servidor en Create

Para completar la validación del lado del cliente, se agregó el parcial `_ValidationScriptsPartial` dentro de la vista `Create.cshtml`. Este parcial permite que los errores se muestren directamente en el navegador, sin esperar primero la respuesta del servidor.

```
1 @section Scripts {
2     <partial name="_ValidationScriptsPartial" />
3 }
```

Listing 5: Views/Producto/Create.cshtml - Scripts de validación

Con esto, el formulario tiene una doble validación. Primero puede mostrar errores desde el cliente y, aun así, el servidor conserva su propia validación como respaldo. Esto es importante porque la validación del navegador puede desactivarse o manipularse, mientras que la validación del servidor controla finalmente si los datos se aceptan o no.



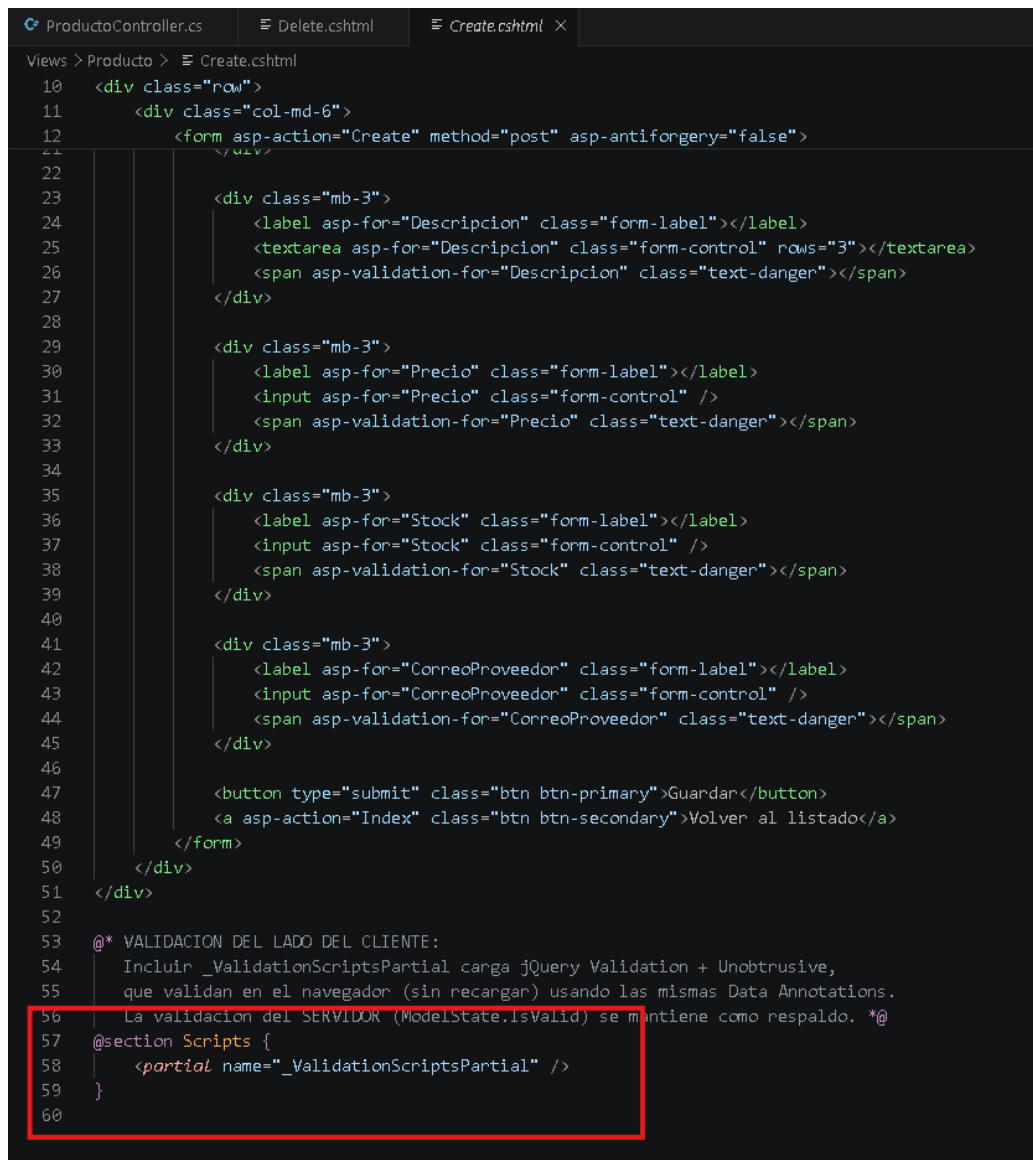
The screenshot shows a web browser at the URL `localhost:5164/Producto/Create`. The application is titled "MiPrimerMVC" and has navigation links for "Home" and "Privacy". The main heading is "Crear Producto". Below the heading, there are five validation error messages in red text:

- El nombre debe tener entre 3 y 100 caracteres.
- La descripcion es obligatoria.
- El precio es obligatorio.
- El stock es obligatorio.
- El correo del proveedor no tiene un formato valido.

The form contains three visible input fields:

- Nombre del producto:** A text input field containing the letter "a". Below it is a red error message: "El nombre debe tener entre 3 y 100 caracteres."
- Descripcion:** A text area input field. Below it is a red error message: "La descripcion es obligatoria."
- Precio (USD):** A text input field. Below it is a red error message: "El precio es obligatorio."

Figura 6: Validación del cliente mostrando errores en el formulario

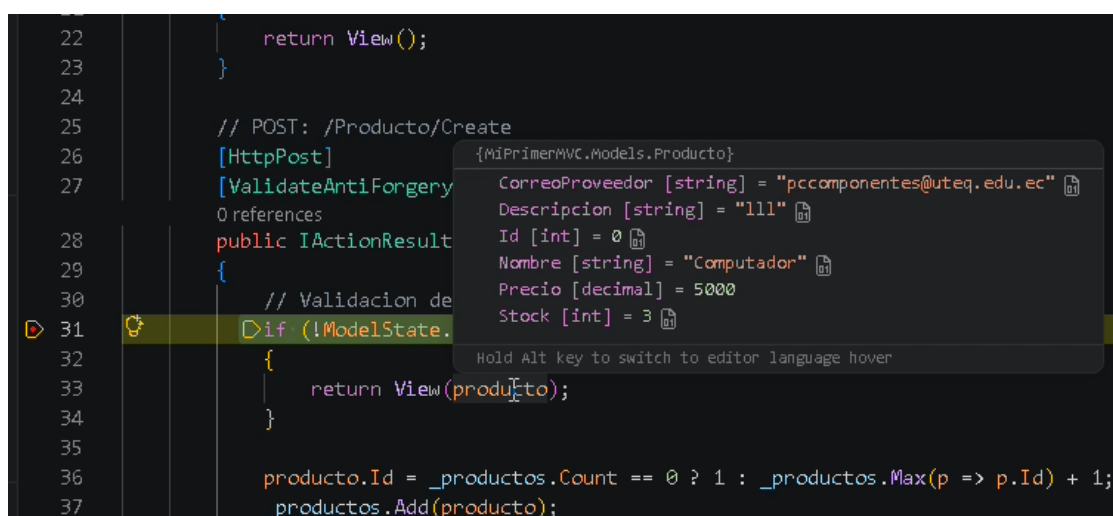


```

10 <div class="row">
11 <div class="col-md-6">
12 <form asp-action="Create" method="post" asp-antiforgery="false">
22 </div>
23 <div class="mb-3">
24 <label asp-for="Descripcion" class="form-label"></label>
25 <textarea asp-for="Descripcion" class="form-control" rows="3"></textarea>
26 <span asp-validation-for="Descripcion" class="text-danger"></span>
27 </div>
28
29 <div class="mb-3">
30 <label asp-for="Precio" class="form-label"></label>
31 <input asp-for="Precio" class="form-control" />
32 <span asp-validation-for="Precio" class="text-danger"></span>
33 </div>
34
35 <div class="mb-3">
36 <label asp-for="Stock" class="form-label"></label>
37 <input asp-for="Stock" class="form-control" />
38 <span asp-validation-for="Stock" class="text-danger"></span>
39 </div>
40
41 <div class="mb-3">
42 <label asp-for="CorreoProveedor" class="form-label"></label>
43 <input asp-for="CorreoProveedor" class="form-control" />
44 <span asp-validation-for="CorreoProveedor" class="text-danger"></span>
45 </div>
46
47 <button type="submit" class="btn btn-primary">Guardar</button>
48 <a asp-action="Index" class="btn btn-secondary">Volver al listado</a>
49 </form>
50 </div>
51 </div>
52
53 @* VALIDACION DEL LADO DEL CLIENTE:
54 Incluir _ValidationScriptsPartial carga jQuery Validation + Unobtrusive,
55 que validan en el navegador (sin recargar) usando las mismas Data Annotations.
56 La validacion del SERVIDOR (ModelState.IsValid) se mantiene como respaldo. *@
57 @section Scripts {
58 <partial name="_ValidationScriptsPartial" />
59 }
60

```

Figura 7: Vista Create.cshtml con ValidationScriptsPartial



```

22 return View();
23 }
24
25 // POST: /Producto/Create
26 [HttpPost]
27 [ValidateAntiForgeryToken]
28 public IActionResult Create([Bind("Id,Nombre,Precio,Stock,CorreoProveedor,Descripcion")] Producto producto)
29 {
30 // Validacion de servidor
31 if (!ModelState.IsValid)
32 {
33 return View(producto);
34 }
35
36 producto.Id = _productos.Count == 0 ? 1 : _productos.Max(p => p.Id) + 1;
37 _productos.Add(producto);

```

Tooltip content:

```

{MiPrimerMVC.Models.Producto}
CorreoProveedor [string] = "pccomponentes@uteq.edu.ec"
Descripcion [string] = "l111"
Id [int] = 0
Nombre [string] = "Computador"
Precio [decimal] = 5000
Stock [int] = 3

```

Figura 8: Validación del servidor mediante ModelState.IsValid

5. Paso 4 – Rama feature/practica-aspnet

La tarea se trabajó en una rama específica llamada **feature/practica-aspnet**. Esto permite separar los cambios de esta práctica del contenido principal del repositorio.

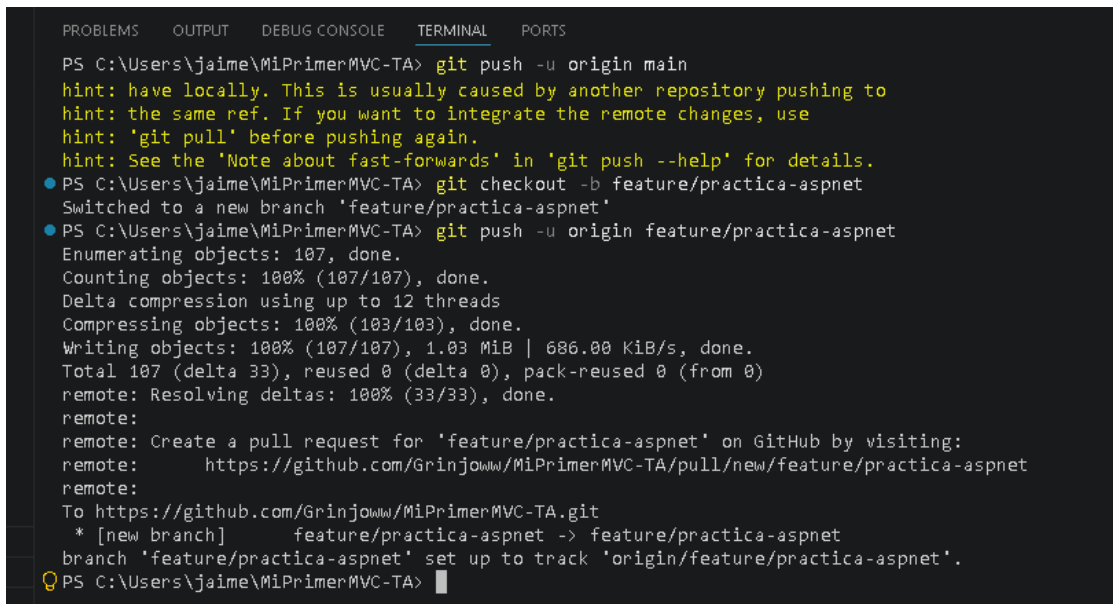
Los comandos utilizados para crear la rama, registrar los cambios y subirlos al repositorio remoto fueron los siguientes:

```
git checkout main
git pull
git checkout -b feature/practica-aspnet

git add .
git commit -m "Extender proyecto ASP.NET Core MVC"
git push -u origin feature/practica-aspnet
```

Listing 6: Flujo básico de Git

El repositorio remoto quedó organizado con la rama solicitada en las directrices de la tarea.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\jaime\MiPrimerMVC-TA> git push -u origin main
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
● PS C:\Users\jaime\MiPrimerMVC-TA> git checkout -b feature/practica-aspnet
Switched to a new branch 'feature/practica-aspnet'
● PS C:\Users\jaime\MiPrimerMVC-TA> git push -u origin feature/practica-aspnet
Enumerating objects: 107, done.
Counting objects: 100% (107/107), done.
Delta compression using up to 12 threads
Compressing objects: 100% (103/103), done.
Writing objects: 100% (107/107), 1.03 MiB | 686.00 KiB/s, done.
Total 107 (delta 33), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (33/33), done.
remote:
remote: Create a pull request for 'feature/practica-aspnet' on GitHub by visiting:
remote:   https://github.com/GrinjoWw/MiPrimerMVC-TA/pull/new/feature/practica-aspnet
remote:
To https://github.com/GrinjoWw/MiPrimerMVC-TA.git
 * [new branch]      feature/practica-aspnet -> feature/practica-aspnet
branch 'feature/practica-aspnet' set up to track 'origin/feature/practica-aspnet'.
● PS C:\Users\jaime\MiPrimerMVC-TA> 
```

Figura 9: Terminal mostrando el flujo de Git y el push de la rama



Figura 10: GitHub mostrando la rama feature/practica-aspnet

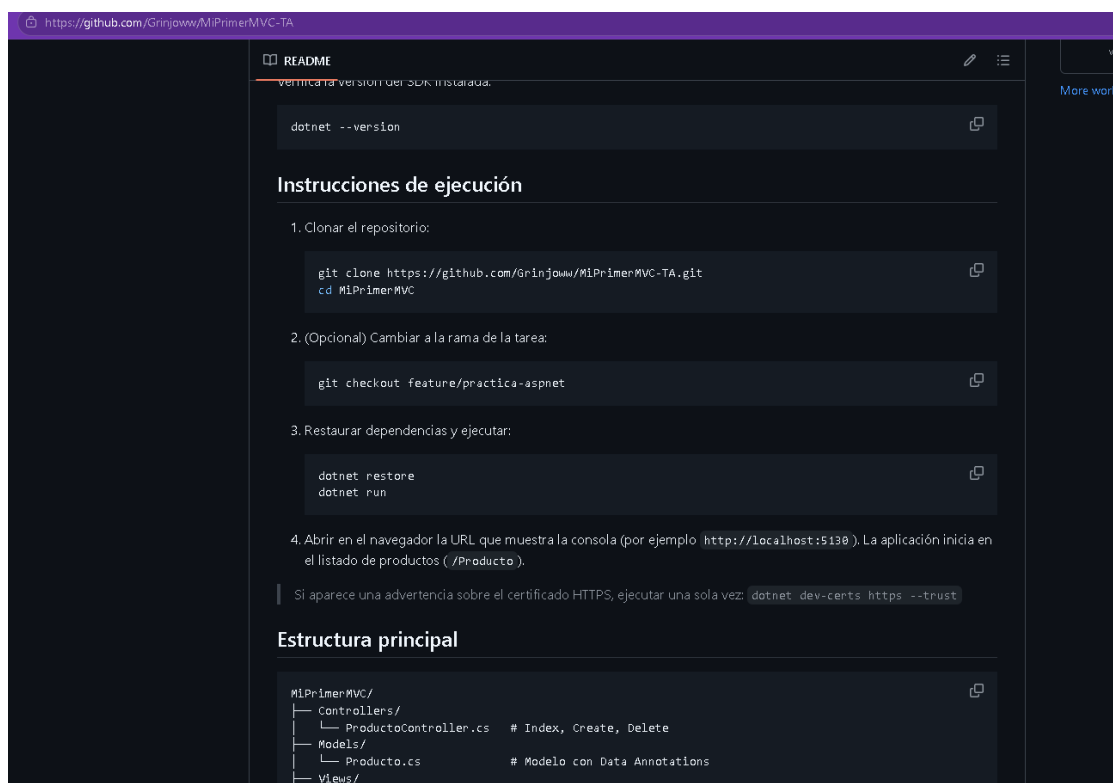


Figura 11: README del repositorio con las instrucciones de ejecución

6. Ejecución y resultados

Para ejecutar la aplicación se utilizó el comando:

```
dotnet run
```

Listing 7: Ejecución del proyecto

Durante la prueba se verificó que el listado de productos se muestra en una tabla HTML, que cada producto cuenta con la acción de eliminación y que el proceso de borrado pasa por una pantalla de confirmación antes de ejecutarse.

También se comprobó que la eliminación se realiza mediante una solicitud POST protegida con token CSRF. Por último, se revisó el formulario de creación para confirmar que la validación funciona tanto en el cliente como en el servidor.

7. Conclusiones

- Se extendió el proyecto ASP.NET Core MVC agregando una tabla HTML para listar los productos registrados.
- Se implementó la funcionalidad de eliminación mediante una acción `Delete` con página de confirmación y envío por POST.
- Se aplicó protección CSRF mediante `AntiForgeryToken` en la vista y `[ValidateAntiForgeryToken]` en el controlador.
- Se mantuvo la validación del servidor con `ModelState.IsValid` y se agregó la validación del cliente mediante `_ValidationScriptsPartial`.
- El trabajo se subió a la rama `feature/practica-aspnet` y se documentó en el archivo `README.md`.